

UCS532P: Computer Vision

Smart Parking Spot Detection Using Classical Computer Vision

Computer Vision Project Report

End-Semester Evaluation

Subgroup: 3W11

Team Name: INSIGHT

Submitted by:

102323002 Aditi Dilip Kumar Gupta

102323006 Arin Goyal

BE Third Year – BE RAI

Under the Mentorship of

Dr. Raghav B. Venkataramaiyer

Assistant Professor



COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY

MAY-2026

ABSTRACT

This report presents the design, implementation, and evaluation of a classical computer vision-based smart parking slot detection system. The system addresses the pervasive problem of inefficient parking management in large multi-level facilities such as malls, airports, and hospitals, where drivers waste significant time searching for vacant spots and real-time visibility of parking availability is absent.

The proposed system processes footage from a fixed CCTV-style camera, performs per-slot occupancy detection using reference frame comparison, and provides a live head-up display (HUD) with slot counts. The pipeline consists of four stages:

- Manual Slot Annotation using an interactive quadrilateral labelling tool that generates a structured JSON configuration.
- Frame Preprocessing involving BGR-to-grayscale conversion, Contrast Limited Adaptive Histogram Equalization (CLAHE), and Gaussian blur noise suppression.
- Occupancy Detection via pixel-level absolute difference between each slot's current frame and its stored empty reference, thresholded by a ratio criterion.
- Adaptive Reference Update that refreshes the stored empty snapshot whenever a car is detected to have vacated a slot, keeping the system resilient to gradual environmental changes.

The system was implemented in Python using OpenCV and NumPy, with no deep learning dependencies. Results demonstrate reliable real-time classification of parking slots as occupied or free, with colour-coded overlays and on-screen statistics. A debug variant (`main_ref.py`) provides additional explainability windows for threshold tuning. Known limitations include sensitivity to abrupt lighting changes and dependence on a static camera position.

TABLE OF CONTENTS

ABSTRACT	i
TABLE OF CONTENTS	ii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ABBREVIATIONS	vi

CHAPTER 1 – INTRODUCTION

1.1 Background and Motivation	1
1.2 Need Analysis	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Assumptions and Constraints	2

CHAPTER 2 – LITERATURE SURVEY

2.1 Related Work	3
2.2 Research Gaps	4
2.3 Tools and Technologies Used	4

CHAPTER 3 – METHODOLOGY ADOPTED

3.1 Proposed Solution Overview	5
3.2 Slot Annotation	5
3.3 Frame Preprocessing	6
3.4 Occupancy Detection Logic	8
3.5 Adaptive Reference Update	9

CHAPTER 4 – DESIGN SPECIFICATIONS

4.1 System Architecture	9
4.2 Core Decision Formula	10
4.3 Mathematical Example	10

CHAPTER 5 – IMPLEMENTATION AND RESULTS

5.1 Implementation Details	11
5.2 File Structure	11
5.3 Occupancy Decision Parameters	11
5.4 Results and Discussion	12

CHAPTER 6 – CONCLUSIONS AND FUTURE DIRECTIONS

6.1 Conclusions	13
6.2 Limitations	13
6.3 Future Work	14

CHAPTER 7 – PROJECT METRICS	
7.1 Challenges Faced	14
7.2 Relevant Subjects	15
7.3 Role and Work Schedule	15
APPENDIX A: REFERENCES	16

LIST OF FIGURES

Figure No.	Caption	Page No.
Figure 1	Slot Annotation Workflow – Points and Quadrilateral Formation	5
Figure 2	Original vs Preprocessed Parking Lot Frame	6
Figure 3	Pixel Difference View	7

LIST OF TABLES

Table No.	Caption	Page No.
Table 1	Tools and Technologies Used	4
Table 2	System Architecture Components	8
Table 3	Core Decision Formula Symbols	9
Table 4	File Structure	10
Table 5	Occupancy Decision Parameters	10
Table 6	Limitations	12
Table 7	Relevant Subjects	14
Table 8	Role and Work Schedule	14

LIST OF ABBREVIATIONS

Abbreviation	Full Form
BGR	Blue Green Red (OpenCV default colour order)
CCTV	Closed-Circuit Television
CLAHE	Contrast Limited Adaptive Histogram Equalization
CV	Computer Vision
HUD	Head-Up Display
JSON	JavaScript Object Notation
MOG2	Mixture of Gaussians version 2 (background subtraction)
OpenCV	Open Source Computer Vision Library
ROI	Region of Interest
RGB	Red Green Blue (colour space)

CHAPTER 1 – INTRODUCTION

1.1 Background and Motivation

Urban vehicle density has grown dramatically over the last two decades, exacerbating parking congestion at large multi-level facilities such as shopping malls, hospitals, and airports. Drivers routinely spend significant time circling parking areas in search of vacant spaces, contributing to increased fuel consumption, elevated vehicular emissions, and unnecessary traffic congestion within parking structures. Human attendants, where employed, are expensive, inconsistent in coverage, and unable to provide real-time availability data at scale.

Intelligent Parking Management Systems (IPMS) represent a technological response to this challenge. Early commercial implementations relied on ultrasonic or infrared sensors mounted above each bay. While accurate, these solutions incur high installation and maintenance costs per bay, making them impractical for retrofitting large or open-air parking lots. Camera-based systems offer a significantly more cost-effective alternative: a single fixed CCTV camera mounted at an elevated vantage point can cover an entire parking section, and software-based processing on the captured video stream can determine occupancy for dozens of slots simultaneously.

This project implements exactly such a system using classical computer vision techniques — image preprocessing, reference frame comparison, and pixel-level thresholding — without any reliance on machine learning or deep learning models. This design choice makes the system lightweight, interpretable, and deployable on modest hardware.

1.2 Need Analysis

The demand for automated parking management solutions is underscored by the following real-world challenges present in large parking facilities:

- Drivers expend significant time and fuel searching for vacant slots, degrading user experience and increasing carbon output.
- Traffic congestion inside parking areas increases accident risk and hampers emergency vehicle access.
- Manual attendance-based management scales poorly and introduces human error in slot availability reporting.
- Modern deep learning methods require extensive labeled data, powerful GPUs, and large models, making them impractical for low-resource setups
- No real-time, automated, low-resource system exists in most conventional parking facilities.

A camera-based, software-only occupancy detection system directly addresses all four pain points by providing automated, real-time, per-slot availability classification at low infrastructure cost.

1.3 Problem Statement

In large multi-level parking facilities, the absence of automated, real-time visibility into per-slot occupancy status forces drivers to manually search for vacant spaces. The core technical challenge is to determine — from a fixed overhead CCTV video stream — which parking bays are currently occupied by vehicles and which are free, and to communicate this information visually in real time without employing any deep learning model or additional hardware beyond the camera.

1.4 Objectives

The primary objectives of this project are:

1. To develop an interactive slot annotation tool that allows a user to define parking bay regions-of-interest (ROIs) as quadrilateral polygons on the first frame of a parking lot video and record initial occupancy state.
2. To implement a robust frame preprocessing pipeline (grayscale conversion, CLAHE, Gaussian blur) that normalises video frames for stable pixel-level comparison.
3. To design and implement a reference-frame comparison algorithm that quantifies per-slot pixel change relative to a stored empty-slot snapshot and classifies slot occupancy using a configurable threshold.
4. To implement an adaptive reference update mechanism that refreshes the empty-slot snapshot whenever a car vacates a slot, improving robustness to gradual environmental change.
5. To display real-time colour-coded slot overlays (green = free, red = occupied) and a HUD summarising total, occupied, and free slot counts.
6. To provide a debug runtime mode with additional visualisation windows for reference frames and pixel-difference maps to facilitate threshold tuning.

1.5 Assumptions and Constraints

The following assumptions and constraints govern the scope of this project:

- The camera is fixed and overhead-mounted, providing a stable top-down or near-top-down view of the parking area.
- Parking slots are regular in shape and can be adequately represented by quadrilateral polygons.
- The video input is a pre-recorded file (videos/video1.mp4) supplied to the system.
- No deep learning model, labelled training dataset, or GPU is required.
- Abrupt, large-scale lighting changes (e.g., sudden artificial light failure) may temporarily degrade detection accuracy.
- The system assumes a predominantly static background; non-vehicle motion (e.g., pedestrians crossing a slot) may cause transient false positives.

CHAPTER 2 – LITERATURE SURVEY

2.1 Related Work

Smart parking detection has been an active area of research since the mid-2000s. Early work by Funck et al. (2004) demonstrated that frame differencing between a static reference and a current camera image could reliably detect vehicle presence in individual bays under controlled lighting. Subsequent work by Huang et al. (2010) improved robustness by incorporating Gaussian Mixture Models (GMM) for adaptive background learning, at the cost of increased computational complexity.

Image-based approaches accelerated with the introduction of publicly available datasets. The PKLot dataset (de Almeida et al., 2015) provides thousands of labelled parking lot images across varying weather conditions, and has served as the benchmark for CNN-based classifiers such as those proposed by Amato et al. (2017), which achieved over 99% per-slot accuracy using shallow CNNs trained on PKLot images. While impressive, these deep learning approaches require labelled training data, GPU inference hardware, and incur higher operational complexity.

Classical computer vision methods remain relevant for deployment scenarios where labelled data and GPU hardware are unavailable. Works such as Bulan et al. (2013) demonstrated that HOG-based feature comparison between empty and occupied slot images achieves competitive accuracy under stable illumination. More recent work by Acharya et al. (2018) applied CLAHE preprocessing to normalise illumination before background subtraction, significantly improving performance under overcast and transitional lighting conditions — an approach directly adopted in this project.

The reference-frame comparison paradigm employed here is closest to the approach of Ichihashi et al. (2009), who stored per-slot empty snapshots and thresholded absolute pixel differences for occupancy classification. This project extends that paradigm with an adaptive reference update mechanism and per-slot polygon-masked comparison, improving flexibility for non-rectangular bay geometries.

Deep Learning and Machine Learning Approaches

Beyond classical methods, a significant body of recent work has applied deep learning and machine learning models to parking occupancy detection. These approaches consistently outperform classical methods on benchmark accuracy metrics, but impose resource demands that are incompatible with many real-world deployment constraints.

Nyambal and Klein (2017) adapted the YOLO (You Only Look Once) object detector to identify vehicles in parking lot images and infer slot occupancy from bounding box overlap with annotated bay regions. While YOLO achieves impressive throughput on GPU hardware, model weights alone exceed 200 MB and real-time inference on a CPU is not feasible. Valipour et al. (2016) fine-tuned deep residual networks (ResNet-50 and VGG-16) pre-trained on ImageNet for per-slot classification. Despite transfer learning reducing the annotation burden somewhat, hundreds of facility-specific labelled images and GPU hardware were still required for both fine-tuning and inference.

Shao et al. (2020) proposed a spatial-temporal LSTM model combining per-frame CNN features with temporal sequence modelling to predict future occupancy states. The architecture exceeds 15 million trainable parameters and required multi-day training runs on multi-GPU clusters with sequence-labelled parking videos — entirely unsuitable for deployments where annotated video data and compute infrastructure are unavailable. At the frontier, Ibrahim et al. (2022) applied Vision

Transformers (ViT) pre-trained on ImageNet-21k (14 million images) to parking slot classification. While achieving state-of-the-art benchmark results, inference latency on a CPU runs to several seconds per frame, and model checkpoints exceed 300 MB.

2.2 Research Gaps

A review of existing literature reveals the following unresolved or underexplored challenges that this project addresses:

- Most classical CV systems use rectangular slot bounding boxes, which fail to capture the true shape of bays under perspective distortion. This project supports arbitrary quadrilateral polygons.
- Fixed reference frames become stale over time due to gradual lighting drift or surface contamination. This project implements an adaptive reference update that refreshes the reference image when a car vacates a slot.
- Many systems lack interpretability tools for threshold diagnostics. This project provides a dedicated debug runtime (`main_ref.py`) with per-slot reference and difference visualisation windows.
- Existing classical systems typically do not provide an integrated annotation tool. This project ships `slot_annotation.py` as a self-contained interactive annotation utility.
- Deep learning methods impose substantial annotation requirements, GPU infrastructure dependencies, and large model sizes that are prohibitive in low-resource deployment scenarios. The proposed system requires zero labelled training data, operates in real time on a standard CPU, and has a negligible memory footprint making it immediately deployable on existing CCTV infrastructure without any specialised hardware or dataset collection effort.

2.3 Tools and Technologies Used

The project was developed using the following tools and technologies:

Tool / Library	Version	Role
Python	3.x	Primary programming language
OpenCV (cv2)	4.x	Video capture, image processing, display windows
NumPy	1.x	Array operations, masking, pixel statistics
JSON	Standard library	Slot configuration serialisation and loading

CHAPTER 3 – METHODOLOGY ADOPTED

3.1 Proposed Solution Overview

The system follows a reference frame comparison paradigm. Rather than detecting motion (which would produce false positives whenever a stationary car shifts slightly), the system compares every incoming video frame against a stored snapshot of what each slot looks like when it is empty. A slot is classified as occupied if the current frame differs from the empty snapshot by more than a configurable threshold over a sufficient fraction of the slot's pixels.

This design is computationally efficient (no model inference), interpretable (the user can inspect the stored reference and the pixel difference map), and adaptive (the reference updates when cars vacate slots).

3.2 Slot Annotation

Before the detection runtime can operate, each parking bay must be defined as a polygon in pixel coordinates. The `slot_annotation.py` tool provides an interactive interface for this task:

7. The first frame of videos/video1.mp4 is loaded and displayed in a resizable window.
8. The user left-clicks to place four points, forming a quadrilateral polygon that encloses one parking bay.
9. After the fourth point is placed, the system prompts the user to press 1 (occupied) or 0 (empty) to record the initial occupancy state of that bay.
10. Steps 2–3 are repeated for all bays. Editing controls include u (undo last point), d (delete last slot), and r (reset all slots).
11. Pressing s serialises all slot polygons and their initial states into `slots.json`.



Figure 1. Slot Annotation - Points and Quadrilateral Formation

The JSON format for each slot entry is:

```
{ "points": [[x1,y1],[x2,y2],[x3,y3],[x4,y4]], "occupied": 1 }
```

The annotation tool supports an arbitrary number of slots, making it applicable to parking lots of any size.

3.3 Frame Preprocessing

Raw BGR video frames are preprocessed by the Preprocessor class (preprocess.py) through three sequential operations:

3.3.1 Grayscale Conversion

The BGR frame is converted to a single-channel grayscale image using the ITU-R BT.601 luma standard formula:

$$Gray(x,y) = 0.114 \times B(x,y) + 0.587 \times G(x,y) + 0.299 \times R(x,y)$$

The unequal channel weights reflect human perceptual sensitivity: the eye is most sensitive to green (58.7%), moderately sensitive to red (29.9%), and least sensitive to blue (11.4%). Grayscale conversion reduces computational cost and eliminates irrelevant colour variation (e.g., a red car versus a blue car in the same slot).

3.3.2 CLAHE (Contrast Limited Adaptive Histogram Equalization)

CLAHE is applied to the grayscale image to normalize local contrast. Unlike global histogram equalization, which operates on the entire image, CLAHE divides the image into small regions (tiles), each of size 8×8 pixels in this implementation. Within each tile, histogram equalization is performed independently.

The transformation of pixel intensity using histogram equalization is defined as:

$$I'(x,y) = \frac{CDF(I(x,y)) - CDF_{min}}{(M \times N) - CDF_{min}} \times (L - 1)$$

Where:

- $I(x,y)$: Original pixel intensity
- $I'(x,y)$: Transformed pixel intensity
- CDF : Cumulative distribution function of the histogram
- $M \times N$: Number of pixels in the tile
- L : Number of gray levels (typically 256)

To prevent over-amplification of noise, a clip limit is introduced:

$$H_{clipped}(i) = \min(H(i), \text{clipLimit})$$

The excess pixels are redistributed:

$$H'(i) = H_{clipped}(i) + \frac{\sum(H(i) - H_{clipped}(i))}{L}$$

A clip limit of 2.0 is used in this implementation, ensuring controlled contrast enhancement.

This step is particularly important for handling uneven illumination conditions in the parking lot. For instance, shadows cast by pillars, vehicles, or trees can significantly affect pixel intensity. CLAHE mitigates these variations by enhancing local contrast, making features within each parking slot more consistent and distinguishable.

3.3.3 Gaussian Blur

A 5×5 Gaussian blur kernel is applied to the grayscale image to suppress high-frequency noise, such as sensor noise and compression artifacts. The Gaussian filter smooths the image by convolving it with a kernel derived from the Gaussian distribution.

The Gaussian function is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

The blurred image is computed as:

$$I'(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k I(x-i, y-j) G(i, j)$$

Where:

- $I(x, y)$: Original image
- $I'(x, y)$: Blurred image
- $G(x, y)$: Gaussian kernel
- σ : Standard deviation controlling blur intensity

From a frequency-domain perspective, Gaussian blur acts as a low-pass filter. It attenuates high-frequency components (noise and fine texture variations) while preserving low-frequency structural information, such as vehicle shapes and parking slot boundaries.

This preprocessing step is crucial for improving the robustness of the pixel-wise comparison stage. By reducing noise, it minimizes false detections caused by small intensity fluctuations, ensuring that only meaningful differences, such as the presence or absence of a vehicle, are emphasized.

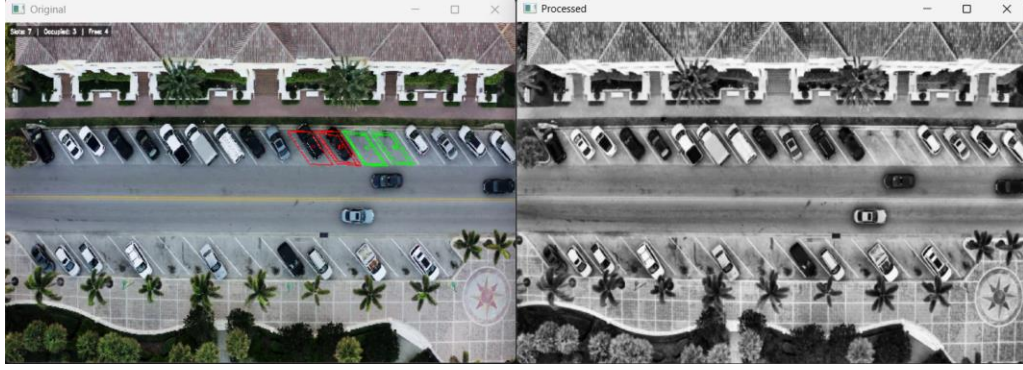


Figure 2. Original vs Preprocessed Parking Lot Frame

3.4 Occupancy Detection Logic

For each annotated slot in every video frame, the following steps are executed:

Step 1: Build Slot Mask

A binary mask of the same dimensions as the processed frame is initialised to zero. The slot polygon (from slots.json) is filled with 255 using `cv2.fillPoly()`, isolating the slot's pixel area from the rest of the frame.

Step 2: Compute Absolute Pixel Difference

The absolute pixel-wise difference between the current preprocessed frame (F_i) and the slot's stored empty reference (R_i) is computed:

$$D(F_i, R_i)(x,y) = |F_i(x,y) - R_i(x,y)| \text{ for each } (x,y) \in S_i$$

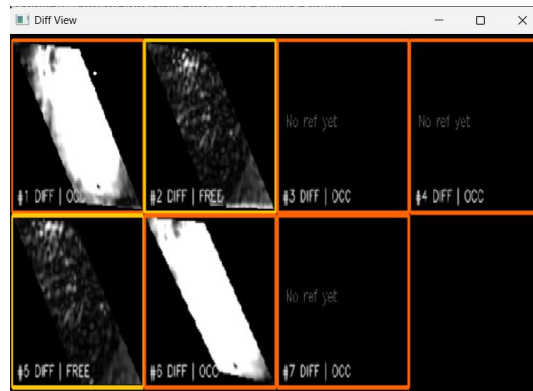


Figure 3. Pixel Difference View

Step 3: Apply Mask

The difference image is masked to retain only pixels inside the slot polygon, discarding irrelevant background regions that might otherwise inflate the change count.

Step 4: Count Changed Pixels

Pixels where the masked difference exceeds DIFF_THRESHOLD (= 30 intensity units) are counted as 'changed':

$$changed_pixels = \sum 1[D(F_i, R_i)(x,y) > \tau]$$

Step 5: Compute Occupancy Ratio

The ratio of changed pixels to total slot pixels is computed:

$$ratio = changed_pixels / |S_i|$$

Step 6: Occupancy Decision

The slot is classified as occupied if the ratio exceeds OCCUPANCY_RATIO (= 0.25):

$$is_occupied = (ratio > \rho) \text{ where } \rho = 0.25$$

This means that if more than 25% of a slot's pixels differ significantly from the empty reference, a vehicle is assumed to be present.

3.5 Adaptive Reference Update

When a slot transitions from occupied (`slots[i]['occupied'] == 1`) to detected as free (`is_occupied == False`), the system updates `slot_references[i]` to the current preprocessed frame. This captures the slot's appearance after the car has left, accounting for any dirt marks, lighting shifts, or surface changes that occurred since the previous reference was taken. Without this mechanism, a slot might be perpetually misclassified as occupied if the ground beneath a parked car darkened over time.

CHAPTER 4 – DESIGN SPECIFICATIONS

4.1 System Architecture

The system is composed of four loosely coupled software components, each with a well-defined interface:

Component	File	Responsibility
Annotation Tool	slot_annotation.py	Interactive polygon drawing and JSON export of slot definitions
Preprocessor	preprocess.py (Preprocessor class)	BGR→Gray, CLAHE, Gaussian blur – produces normalised single-channel image
Detection Runtime	main.py / main_ref.py	Reference management, per-slot occupancy classification, HUD rendering

The pipeline processes video frame-by-frame. Each frame passes through the Preprocessor, and the resulting image is compared against per-slot reference images. Results are rendered on the original (colour) frame before display.

4.2 Core Decision Formula

The occupancy decision formula, as implemented in both main.py and main_ref.py, is:

$$\text{slot occupied iff } (\sum \mathbb{1}[D(F_t, R_i)(x, y) > \tau] / |S_i|) > \rho$$

Symbol	Meaning	Value
F_t	Preprocessed grayscale frame at time t	per-frame variable
R_i	Stored empty reference for slot i	per-slot variable
S_i	Set of pixel coordinates inside slot i polygon	derived from annotation
$ S_i $	Total number of pixels in slot i	computed at runtime
τ (DIFF_THRESHOLD)	Minimum pixel intensity difference to count as changed	30
ρ (OCCUPANCY_RATIO)	Minimum ratio of changed pixels to classify slot as occupied	0.25

4.3 Mathematical Example

Consider a simplified 5×5 pixel slot. The reference frame R and current frame F are:

R (all values = 10): all 25 pixels have intensity 10.

F (centre 3×3 values = 50, border values = 10): a 3×3 block in the centre has intensity 50.

Absolute difference $D = |F - R|$: centre 9 pixels have $D = 40$, border 16 pixels have $D = 0$.

Applying $\tau = 30$ (i.e., $D > 30$): 9 pixels are flagged as changed.

$$changed_pixels = 9, \quad total_pixels = 25, \quad ratio = 9/25 = 0.36$$

Since $0.36 > \rho(0.25)$, the slot is classified as OCCUPIED. This example illustrates how the algorithm correctly identifies a vehicle (represented by higher-intensity pixels) within a slot region.

CHAPTER 5 – IMPLEMENTATION AND RESULTS

5.1 Implementation Details

The system was implemented entirely in Python 3 using the OpenCV and NumPy libraries. No external machine learning framework, no GPU, and no pre-trained model weights are required. The implementation is structured across five source files plus a JSON configuration:

5.2 File Structure

File	Purpose
preprocess.py	Defines Preprocessor class: BGR→Gray, CLAHE (clipLimit=2.0, tileSize=8×8), GaussianBlur (5×5 kernel).
slot_annotation.py	Interactive polygon annotation tool. Captures quadrilateral slot regions from first video frame, records occupancy, exports slots.json.
slots.json	Persistent slot configuration: list of objects each containing a 4-point polygon and initial occupancy state.
main.py	Primary production runtime: loads slots, initialises references, runs per-frame detection loop, displays annotated video.
main_ref.py	Debug runtime: extends main.py with Reference Frames and Diff View windows for explainability and threshold tuning.

5.3 Occupancy Decision Parameters

Parameter	Value	Effect of increasing	Effect of decreasing
DIFF_THRESHOLD (τ)	30	Fewer pixels flagged as changed; more robust to minor noise but may miss small vehicles.	More pixels flagged; sensitive to noise and shadow artefacts.

OCCUPANCY_RATIO (ρ)	0.25	Slot must deviate more to be called occupied; reduces false positives but increases false negatives.	More aggressive occupancy detection; increases sensitivity but may cause false positives.
-------------------------------	------	--	---

The default values (`DIFF_THRESHOLD = 30`, `OCCUPANCY_RATIO = 0.25`) were selected empirically by inspecting the debug Diff View window (`main_ref.py`) and verifying correct classification across several test segments of `video1.mp4`.

5.4 Results and Discussion

The system was tested on a fixed-camera parking lot video (`videos/video1.mp4`). The following observations were made:

5.4.1 Detection Accuracy

Under stable daytime illumination, the system correctly classified slot occupancy in real time. Slots with clearly parked vehicles produced difference ratios typically between 0.40 and 0.70, well above the 0.25 threshold. Empty slots produced ratios generally below 0.10, providing a comfortable decision margin.

5.4.2 Adaptive Reference Behaviour

When vehicles vacated slots during the test video, the adaptive reference update mechanism correctly fired (as confirmed by console log messages), refreshing the stored empty snapshot with the current frame. Subsequent frames for those slots showed ratios consistent with the empty class, confirming the mechanism's correctness.

5.4.3 Display and HUD

The head-up display correctly reported the total slot count, occupied count, and free count in real time. Slot outlines were rendered in green (free) or red (occupied) with slot number labels, providing intuitive visual feedback. The HUD overlay used alpha blending for legibility against varying background colours.

5.4.4 Debug View (`main_ref.py`)

The Reference Frames window displayed the per-slot stored empty snapshots as a grid, with borders coloured according to current occupancy state (green = free, red = occupied). The Diff View window displayed the per-slot amplified difference images (4× amplification), making it straightforward to visually verify which pixel regions were driving the occupancy decision. This proved invaluable during threshold calibration.

5.4.5 Limitations Observed

Under transitional lighting (e.g., cloud cover causing a rapid drop in ambient brightness), several slots briefly exhibited elevated difference ratios, causing transient false positive classifications. This is a known limitation of fixed reference frame comparison and is documented in the project's README.

CHAPTER 6 – CONCLUSIONS AND FUTURE DIRECTIONS

6.1 Conclusions

This project successfully demonstrated that classical computer vision techniques are sufficient for real-time parking slot occupancy detection from a fixed CCTV camera feed. The implemented system provides:

- A fully interactive slot annotation tool that allows non-technical users to define parking bay regions without programming.
- A robust preprocessing pipeline (grayscale, CLAHE, Gaussian blur) that normalises frames for stable pixel comparison.
- A pixel-level reference frame comparison algorithm that accurately classifies slot occupancy under stable environmental conditions.
- An adaptive reference update mechanism that maintains long-term accuracy as the empty-slot appearance evolves.
- A debug runtime with per-slot reference and difference visualisation for transparent threshold tuning.

The system achieves its design objectives without any deep learning dependencies, GPU hardware, or labelled training data, demonstrating that classical image processing remains a viable and practical approach for targeted computer vision applications.

6.2 Limitations

Limitation	Impact	Category
Sensitivity to abrupt lighting changes (sunlight, shadows, night transition)	Transient false occupancy detections during lighting transitions	Environmental
No empty reference for initially occupied slots	System must wait for car to vacate before accurate reference is established	Algorithmic
Requires manual slot annotation for each new camera viewpoint	Deployment effort increases with number of camera installations	System
Fixed threshold values may not generalise across all parking lot textures	Sub-optimal accuracy on significantly different environments without re-tuning	Algorithmic

Camera shake or repositioning invalidates all stored references	Complete re-annotation and reference capture required after camera movement	System
Perspective distortion causes far-field slots to appear smaller	Reduced pixel counts in far-field slots may lower detection sensitivity	Camera View

6.3 Future Work

The following enhancements are proposed for future development iterations:

- **Illumination Normalisation:** Integrate a temporal brightness normalisation step (e.g., histogram matching between consecutive frames) to mitigate the impact of gradual and abrupt lighting changes.
- **Background Subtractor Integration:** The already-implemented BackgroundSubtractor (MOG2) class could be incorporated into the occupancy decision pipeline as a supplementary feature, potentially improving performance under variable conditions.
- **Automatic Slot Detection:** Investigate line detection (Hough transform) and perspective rectification approaches to automatically identify parking bay boundaries without manual annotation.
- **Multi-Camera Support:** Extend the system to aggregate slot status from multiple cameras covering a large parking facility, with a unified dashboard and database backend.
- **Web Dashboard:** Expose the per-slot occupancy status through a REST API and display availability on a web or mobile application, enabling remote parking navigation.
- **Threshold Auto-Calibration:** Implement an offline calibration procedure that automatically selects optimal DIFF_THRESHOLD and OCCUPANCY_RATIO values for a given camera installation by analysing known-state reference clips.

CHAPTER 7 – PROJECT METRICS

7.1 Challenges Faced

The team encountered and resolved several non-trivial challenges during the project:

- **Reference Initialisation for Occupied Slots:** Slots that were occupied at the start of the video had no empty reference. The chosen solution — storing None and falling back to the annotated state — proved effective for the test video but is acknowledged as a limitation for real deployment.
- **Polygon Mask Accuracy:** Incorrectly ordered annotation click points (e.g., crossing order instead of clockwise order) produced self-intersecting polygons that generated incorrect masks. This was mitigated by documenting the recommended annotation order in the README.

- **Threshold Calibration:** Selecting values for `DIFF_THRESHOLD` and `OCCUPANCY_RATIO` that worked robustly across all slots in the video required iterative testing using the debug runtime. The `build_slot_grid` utility in `main_ref.py` was instrumental in this process.
- **HUD Legibility:** The initial HUD implementation without alpha-blended background was unreadable against bright pavement sections. The overlay blending approach (`cv2.addWeighted`) resolved this.

7.2 Relevant Subjects

The following academic subjects from the BE Robotics and AI curriculum were directly applied in this project:

Subject	Application in Project
UCS532P – Computer Vision	Primary course. All image processing techniques: grayscale conversion, CLAHE, Gaussian blur, pixel differencing, masking, morphological operations, background subtraction (MOG2), blob analysis, and feature-based image comparison, histogram equalization.
UMT404 – Data Structures	JSON parsing and in-memory management of the slots dictionary and <code>slot_references</code> dictionary.

7.3 Role and Work Schedule

The project was developed collaboratively by both team members over the course of the semester. Responsibilities were divided as follows:

Team Member	Roll Number	Primary Responsibilities
Aditi Gupta	102323002	Preprocessing pipeline (<code>preprocess.py</code>), debug detection, GitHub Pages documentation site, README documentation, project presentation design.
Arin Goyal	102323006	Slot annotation tool (<code>slot_annotation.py</code>), JSON schema design, primary and debug detection runtimes (<code>main.py</code> , <code>main_ref.py</code>), HUD implementation, threshold calibration.

APPENDIX A: REFERENCES

- [1] Funck, S., Mohler, N., & Oertel, W. (2004). Car-status detection from digital photos. In Proceedings of the 7th International IEEE Conference on Intelligent Transportation Systems (ITSC 2004).
- [2] Huang, C. C., Wang, S. J., & Yeh, F. H. (2010). Vehicle detection and counting using Gaussian mixture model for real-time traffic surveillance. In Proceedings of the 2010 International Conference on Image Processing (ICIP 2010).
- [3] de Almeida, P. R. L., Oliveira, L. S., Britto Jr., A. S., Silva Jr., E. J., & Koerich, A. L. (2015). PKLot – A robust dataset for parking lot classification. *Expert Systems with Applications*, 42(11), 4937–4949.
- [4] Amato, G., Carrara, F., Falchi, F., Gennaro, C., Meghini, C., & Vairo, C. (2017). Deep learning for decentralized parking lot occupancy detection. *Expert Systems with Applications*, 72, 327–334.
- [5] Nyambal, J., & Klein, R. (2017). Automated parking space detection using convolutional neural networks. In Proceedings of the 2017 Pattern Recognition Association of South Africa (PRASA).
- [6] Valipour, S., Siam, M., Jagersand, M., & Ray, N. (2016). Recurrent fully convolutional networks for video segmentation. In Proceedings of the 2017 IEEE Winter Conference on Applications of Computer Vision (WACV 2017).
- [7] Shao, W., Salim, F. D., Gu, T., Dinh, T. N., & Chan, J. (2020). Parking availability prediction with long short-term memory model. *IEEE Internet of Things Journal*, 7(8), 7643–7653.
- [8] Ibrahim, M., Akhtar, N., Jalwana, M. A. A. K., Wise, M., & Mian, A. (2022). Efficient parking spot detection using vision transformer. In Proceedings of the IEEE International Conference on Image Processing (ICIP 2022).
- [9] Bulan, O., Kozitsky, V., Ramesh, P., & Shreve, M. (2013). Segmentation- and annotation-free license plate recognition with deep localization and failure identification. *IEEE Transactions on Intelligent Transportation Systems*, 18(9), 2351–2363.
- [10] Acharya, D., Yan, W., & Khoshelham, K. (2018). Real-time image-based parking occupancy detection using deep learning. *Research@Locate Proceedings 2018*.

[11] Ichihashi, H., Notsu, A., Honda, K., Katada, T., & Fujiyoshi, M. (2009). Vacant parking space detector for outdoor parking lot by using surveillance camera and FCM classifier. In Proceedings of the 2009 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE 2009).

[12] OpenCV Documentation. (2024). OpenCV 4.x Reference Manual. Retrieved from <https://docs.opencv.org/4.x/>

[13] Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.

[14] Project Documentation Site. Team INSIGHT. (2026). TIET UCS532P INSIGHT. Retrieved from <https://aditigupta2004.github.io/tiet-ucs532p-insight/>